

Factory Inventory Management System

A two-tier demo application: a Vue 3 single-page frontend talking to a Python FastAPI backend that serves filtered views over an in-memory JSON dataset. No database, no auth — designed for workshop and prototype use.

System Architecture

Layered view from browser to data files. Each layer talks only to the one beneath it.

PRESENTATION

Vue 3 SPA · port 3000

7 Views (Dashboard, Inventory, Orders, Spending, Demand, Backlog, Reports)

9 Components (FilterBar, modals, ProfileMenu)

3 Composables (useAuth, useFilters, useI18n)

↓ axios over HTTP

API CLIENT

client/src/api.js

Centralized axios wrapper. Builds query strings from filter state, returns parsed JSON. Single base URL:
`http://localhost:8001/api`

↓ HTTP request

API SERVER

FastAPI · port 8001

CORS middleware (allow_origins=*)

Pydantic request/response models

`apply_filters() + filter_by_month()`

Swagger docs at `/docs`

↓ in-memory lookup

DATA

`server/mock_data.py → server/data/*.json`

`inventory.json`

`orders.json`

`demand_forecasts.json`

`backlog_items.json`

`spending.json`

`transactions.json`

`purchase_orders.json`

Loaded once at startup. Mutations are not persisted — restart resets all state.

Tech Stack

Production dependencies pinned in `package.json` and `pyproject.toml`.

● Frontend

Browser SPA, served by Vite dev server

Vue ^3.4.21

Vue Router ^4.3.0

Axios ^1.6.7

Vite ^5.2.0

● Backend

HTTP API, runs on uvicorn

Python >=3.11

FastAPI >=0.110

Uvicorn >=0.24

Pydantic >=2.5

● Data Layer

No database — JSON files in memory

JSON files server/data/

Loader mock_data.py

Validation Pydantic models

● Tooling

Dev, build, and test

uv Python env / deps

npm Node deps

pytest backend tests

Persistence

none

scripts/start.sh

one-shot launcher

Data Flow

How a user-initiated filter change reaches the data files and back.

1

User changes filter

FilterBar component updates a ref in useFilters composable.

2

View reacts

A watch in the active view fires and calls the relevant api.* method.

3

HTTP request

api.js builds URL params and sends an axios GET to :8001/api/*.

4

FastAPI filters

Endpoint runs apply_filters() and (for orders) filter_by_month().

5

Pydantic validates

Result list is serialized through the response_model and returned as JSON.

6

Vue re-renders

Response stored in a ref. Computed properties recalculate. Template updates.

API Endpoints

METHOD	PATH	PURPOSE	FILTERS
GET	/api/inventory	List inventory items	warehouse, category
GET	/api/inventory/{id}	Single item lookup	—
GET	/api/orders	List orders	warehouse, category, status, month
GET	/api/orders/{id}	Single order lookup	—
GET	/api/demand	Demand forecasts	—

METHOD	PATH	PURPOSE	FILTERS
GET	/api/backlog	Backlog with PO status	—
GET	/api/dashboard/summary	Aggregated KPIs	all four filters
GET	/api/spending/summary	Spending totals	—
GET	/api/spending/monthly	Monthly spend breakdown	—
GET	/api/spending/categories	Spend by category	—
GET	/api/spending/transactions	Recent transactions	—
GET	/api/reports/quarterly	Quarterly performance	—
GET	/api/reports/monthly-trends	Month-over-month trends	—
POST	/api/purchase-orders	Create a PO for a backlog item	—

Generated from source: `server/main.py`, `client/src/api.js`, `package.json`, `pyproject.toml`

Frontend `:3000` · Backend `:8001` · API docs `:8001/docs`